

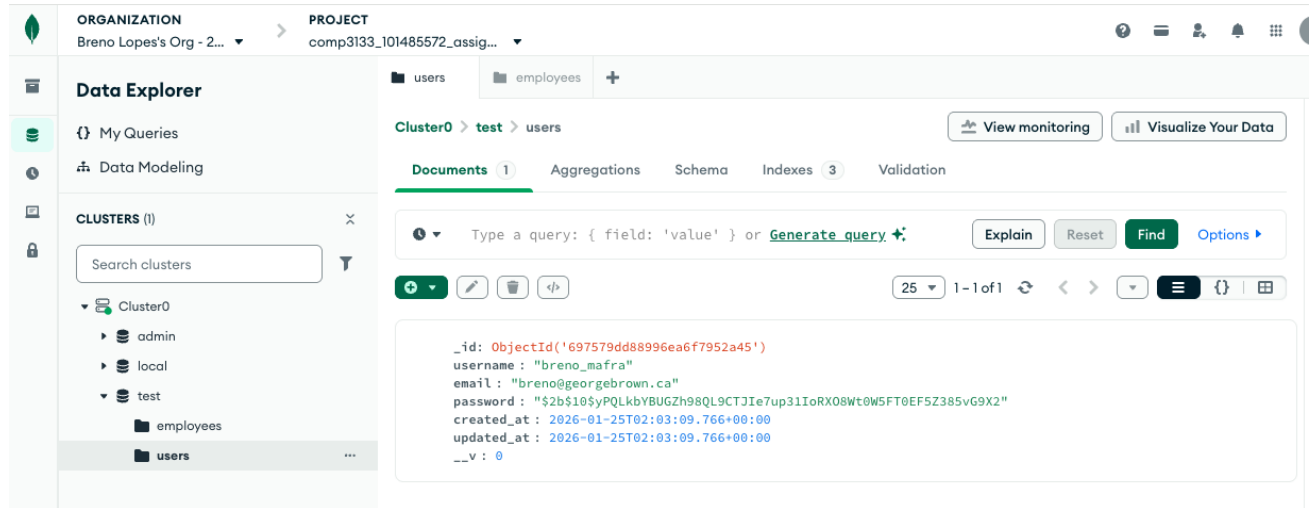
Student Name: Breno Lopes Mafra

Student ID: 101485572

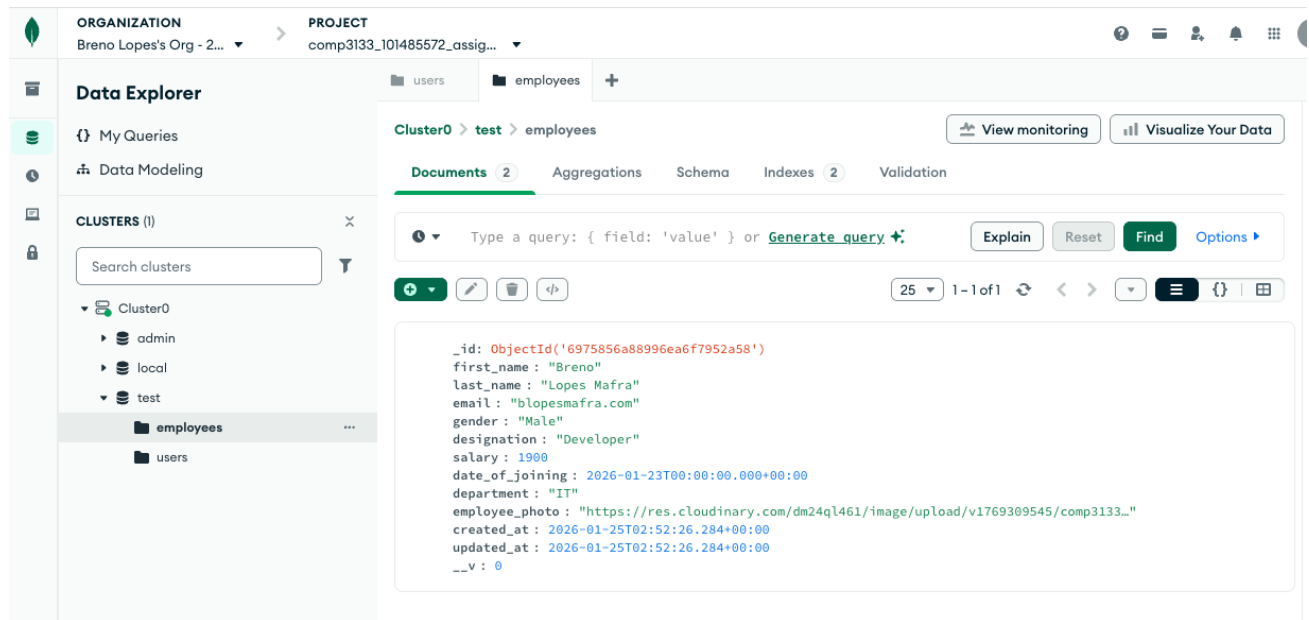
Assignment 1 – Screenshots

1. MongoDB Console Screenshots:

“Users” collection:



“Employees” collection:



2. Postman API Collection (.json file also attached in submission):

https://comp3095-f2025.postman.co/workspace/comp3133_101485572_assignment1~6e872b7b-142b-43b1-949a-7b7399d4bab3/collection/40211743-d7d7ed38-36a9-43ae-b1d4-58e4ab034525?action=share&source=copy-link&creator=40211743

3. Screenshots:

Signup:

The screenshot shows the GraphQL Playground interface. On the left, the 'Collections' sidebar lists various queries under 'Employee' and 'User' categories. The 'User' collection is expanded, showing 'POST Signup' and 'POST Login'. The main editor area is set to 'POST' with the URL 'http://localhost:4000/graphql'. The query is a GraphQL mutation to create a user. The response shows a successful signup with a 200 OK status.

Query:

```
1 mutation CreateUser($username: String!, $email: String!,  
2   $password: String!) {  
3   signup(username: $username, email: $email, password:  
4     $password) {  
5     id  
6     username  
7     email  
8   }  
9 }
```

GraphQL Variables:

```
1 {  
2   "username": "breno_mafra",  
3   "email": "breno@georgebrown.ca",  
4   "password": "12345678"  
5 }
```

Response (JSON):

```
1 {  
2   "data": {  
3     "signup": {  
4       "id": "697579dd88996ea6f7952a45",  
5       "username": "breno_mafra",  
6       "email": "breno@georgebrown.ca"  
7     }  
8   }  
9 }
```

Trying to sign up with same credentials:

The screenshot shows the GraphQL Playground interface with the same query as the previous screenshot. However, the response shows an error because the username or email already exists. The status is 200 OK, but the data field is null.

Query:

```
1 mutation CreateUser($username: String!, $email: String!,  
2   $password: String!) {  
3   signup(username: $username, email: $email, password:  
4     $password) {  
5     id  
6     username  
7     email  
8   }  
9 }
```

GraphQL Variables:

```
1 {  
2   "username": "breno_mafra",  
3   "email": "breno@georgebrown.ca",  
4   "password": "12345678"  
5 }
```

Response (JSON):

```
1 {  
2   "errors": [  
3     {  
4       "message": "Username or Email already exists."  
5     }  
6   ],  
7   "data": {  
8     "signup": null  
9   }  
10 }
```

Login:

The screenshot shows the GraphQL Playground interface. On the left sidebar, the 'User' collection is expanded, and the 'Login' endpoint is selected. The main area displays a POST request to `http://localhost:4000/graphql`. The query is:

```
1 query UserLogin($usernameOrEmail: String!, $password: String!) {
2   login(usernameOrEmail: $usernameOrEmail, password: $password)
3 }
```

The GraphQL variables are:

```
1 {
2   "usernameOrEmail": "breno_mafra",
3   "password": "12345678"
4 }
```

The response status is 200 OK. The JSON response body is:

```
1 {
2   "data": {
3     "login": "Login successful!"
4   }
5 }
```

Trying to login with wrong password:

The screenshot shows the GraphQL Playground interface with the same setup as the previous one, but with a different password. The query is:

```
1 query UserLogin($usernameOrEmail: String!, $password: String!) {
2   login(usernameOrEmail: $usernameOrEmail, password: $password)
3 }
```

The GraphQL variables are:

```
1 {
2   "usernameOrEmail": "breno_mafra",
3   "password": "876454321"
4 }
```

The response status is 200 OK. The JSON response body is:

```
1 {
2   "errors": [
3     {
4       "message": "Invalid password. Please try again."
5     }
6   ],
7   "data": {
8     "login": null
9   }
10 }
```

Trying to login with wrong username/email:

The screenshot shows the GraphQL Playground interface. The left sidebar displays a collection named 'Employee' with various endpoints, and a 'User' collection with 'SignUp' and 'Login' endpoints. The main editor is set to a POST request to 'http://localhost:4000/graphql'. The query is:

```
1 query UserLogin($usernameOrEmail: String!, $password: String!) {
2   login(usernameOrEmail: $usernameOrEmail, password: $password)
3 }
```

The GraphQL variables are:

```
1 {
2   "usernameOrEmail": "mafra_breno",
3   "password": "12345678"
4 }
```

The response status is 200 OK. The JSON response shows an error:

```
1 {
2   "errors": [
3     {
4       "message": "User not found. Please check your credentials."
5     }
6   ],
7   "data": {
8     "login": null
9   }
10 }
```

Add new employee:

The screenshot shows the GraphQL Playground interface with the 'POST Add new Em' endpoint selected. The query is:

```
1 mutation CreateEmployee($first_name: String!, $last_name: String!, $email: String!, $gender: String!, $designation: String!, $salary: Float!, $date_of_joining: String!, $department: String!, $employee_photo: String!) {
2   addEmployee(
3     first_name: $first_name,
4     last_name: $last_name,
5     email: $email,
6     gender: $gender,
7     designation: $designation,
8     salary: $salary,
9     date_of_joining: $date_of_joining,
10    department: $department,
11    employee_photo: $employee_photo
12  ) {
13    id
14    first_name
15    employee_photo
16  }
17 }
```

The GraphQL variables are:

```
1 {
2   "first_name": "Breno",
3   "last_name": "Lopes Mafra",
4   "email": "breno@lopesmafra.com",
5   "gender": "Male",
6   "designation": "Developer",
7   "salary": 1600,
8   "date_of_joining": "2026-01-23",
9   "department": "IT",
10  "employee_photo": "https://conteudo.imguol.com.br/3f/2020/11/17/kid-bengala-via-instagram-1605617856164_v2_4x3.jpg"
11 }
```

The response status is 200 OK. The JSON response shows the created employee:

```
1 {
2   "data": {
3     "addEmployee": {
4       "id": "69757aee88996ea6f7952a50",
5       "first_name": "Breno",
6       "employee_photo": "https://res.cloudinary.com/dm24ql461/image/upload/v1769306862/comp3133_assignment1/c0duksysibqurlyumpv.jpg"
7     }
8   }
9 }
```

Trying to register employee with salary < 1000

The screenshot shows the GraphQL Playground interface. The left sidebar contains a 'Collections' panel with 'Employee' and 'User' collections. The 'Employee' collection is expanded, showing a 'POST Add new Employee' endpoint. The main area shows a 'POST' request to 'http://localhost:4000/graphql'. The query is a mutation to create an employee with a salary of 900. The response is a 200 OK status with a JSON body indicating a validation error: 'Employee validation failed: salary: Path `salary` (900) is less than minimum allowed value (1000)'. The GraphQL variables are set to create an employee named Breno, a Developer, with a salary of 900, joining on 2026-01-23, in the IT department, with a specific employee photo URL.

```
mutation CreateEmployee($first_name: String!,  
  $last_name: String!, $email: String!, $gender:  
  String!, $designation: String!, $salary: Float!,  
  $date_of_joining: String!, $department: String!,  
  $employee_photo: String!) {  
  addEmployee(  
    first_name: $first_name,  
    last_name: $last_name,  
    email: $email,  
    gender: $gender,  
    designation: $designation,  
    salary: $salary,  
    date_of_joining: $date_of_joining,  
    department: $department,  
    employee_photo: $employee_photo  
  ) {  
    id  
    first_name  
    employee_photo  
  }  
}
```

```
{  
  "errors": [  
    {  
      "message": "Employee validation failed: salary: Path `salary` (900) is less than minimum allowed value (1000)."  
    }  
  ],  
  "data": {  
    "addEmployee": null  
  }  
}
```

Trying to register employee with an existing email:

The screenshot shows the GraphQL Playground interface. The left sidebar contains a 'Collections' panel with 'Employee' and 'User' collections. The 'Employee' collection is expanded, showing a 'POST Add new Employee' endpoint. The main area shows a 'POST' request to 'http://localhost:4000/graphql'. The query is a mutation to create an employee with a salary of 1900. The response is a 200 OK status with a JSON body indicating a duplicate key error: 'E11000 duplicate key error collection: test.employees index: email_1 dup key: { email: \"breno@lopesmafra.com\" }'. The GraphQL variables are set to create an employee named Breno, a Developer, with a salary of 1900, joining on 2026-01-23, in the IT department, with a specific employee photo URL.

```
mutation CreateEmployee($first_name: String!,  
  $last_name: String!, $email: String!, $gender:  
  String!, $designation: String!, $salary: Float!,  
  $date_of_joining: String!, $department: String!,  
  $employee_photo: String!) {  
  addEmployee(  
    first_name: $first_name,  
    last_name: $last_name,  
    email: $email,  
    gender: $gender,  
    designation: $designation,  
    salary: $salary,  
    date_of_joining: $date_of_joining,  
    department: $department,  
    employee_photo: $employee_photo  
  ) {  
    id  
    first_name  
    employee_photo  
  }  
}
```

```
{  
  "errors": [  
    {  
      "message": "E11000 duplicate key error collection: test.employees index: email_1 dup key: { email: \"breno@lopesmafra.com\" }"  
    }  
  ],  
  "data": {  
    "addEmployee": null  
  }  
}
```

Trying to register an employee with a non-listed gender:

The screenshot shows the GraphQL Playground interface. The left sidebar displays the 'Employee' collection with various endpoints. The main editor shows a POST request to `http://localhost:4000/graphql`. The query is a mutation to create a new employee with the following variables:

```
mutation CreateEmployee($first_name: String!, $last_name: String!, $email: String!, $gender: String!, $designation: String!, $salary: Float!, $date_of_joining: String!, $department: String!, $employee_photo: String!) {
  addEmployee(
    first_name: $first_name,
    last_name: $last_name,
    email: $email,
    gender: $gender,
    designation: $designation,
    salary: $salary,
    date_of_joining: $date_of_joining,
    department: $department,
    employee_photo: $employee_photo
  ) {
    id
    first_name
    employee_photo
  }
}
```

The response shows a 200 OK status with the following JSON body:

```
{
  "errors": [
    {
      "message": "Employee validation failed: gender: 'Alien' is not a valid enum value for path 'gender'."
    }
  ],
  "data": {
    "addEmployee": null
  }
}
```

Trying to register an employee with empty username/email:

The screenshot shows the GraphQL Playground interface. The left sidebar displays the 'Employee' collection with various endpoints. The main editor shows a POST request to `http://localhost:4000/graphql`. The query is a mutation to create a new employee with the following variables:

```
mutation CreateEmployee($first_name: String!, $last_name: String!, $email: String!, $gender: String!, $designation: String!, $salary: Float!, $date_of_joining: String!, $department: String!, $employee_photo: String!) {
  addEmployee(
    first_name: $first_name,
    last_name: $last_name,
    email: $email,
    gender: $gender,
    designation: $designation,
    salary: $salary,
    date_of_joining: $date_of_joining,
    department: $department,
    employee_photo: $employee_photo
  ) {
    id
    first_name
    employee_photo
  }
}
```

The response shows a 200 OK status with the following JSON body:

```
{
  "errors": [
    {
      "message": "Employee validation failed: first_name: Path 'first_name' is required."
    }
  ],
  "data": {
    "addEmployee": null
  }
}
```

Trying to register an employee with an empty department:

comp3133_101485572_assignment1 New Import

POST SignUp POST Login POST Add new En GET Untitled Reu GET Get all employ + No environment

Employee / Add new Employee Save Share

POST http://localhost:4000/graphql Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL Auto Fetch Schema Fetched

QUERY

```
1 mutation CreateEmployee($first_name: String!,
2   $last_name: String!, $email: String!, $gender:
3   String!, $designation: String!, $salary: Float!,
4   $date_of_joining: String!, $department: String!,
5   $employee_photo: String!) {
6   addEmployee(
7     first_name: $first_name,
8     last_name: $last_name,
9     email: $email,
10    gender: $gender,
11    designation: $designation,
12    salary: $salary,
13    date_of_joining: $date_of_joining,
14    department: $department,
15    employee_photo: $employee_photo
16  ) {
17    id
18    first_name
19    employee_photo
20  }
21 }
```

GRAPHQL VARIABLES

```
1 {
2   "first_name": "Breno",
3   "last_name": "Lopes Mafra",
4   "email": "b@lopesmafra.com",
5   "gender": "Male",
6   "designation": "Developer",
7   "salary": 1900,
8   "date_of_joining": "2026-01-23",
9   "department": "",
10  "employee_photo": "https://
11  conteudo.imguol.com.br/3f/
12  2020/11/17/
13  kid-bengala-via-instagram-1605
14  617856164_v2_4x3.jpg"
15 }
```

Body Cookies Headers (8) Test Results 200 OK 925 ms 394 B Save Response

JSON Preview Visualization

```
1 {
2   "errors": [
3     {
4       "message": "Employee validation failed: department: Path `department` is required."
5     }
6   ],
7   "data": {
8     "addEmployee": null
9   }
10 }
```

Get all employees:

comp3133_101485572_assignment1 New Import

POST SignUp POST Login POST Add new En GET Untitled Reu POST Get all empk + No environment

Employee / Get all employees Save Share

POST http://localhost:4000/graphql Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL Auto Fetch Schema Fetched

QUERY

```
1 query {
2   getAllEmployees {
3     id
4     first_name
5     last_name
6     email
7     employee_photo
8   }
9 }
```

GRAPHQL VARIABLES

```
1 {
2   "first_name": "Breno",
3   "last_name": "Lopes Mafra",
4   "email": "b@lopesmafra.com",
5   "gender": "Male",
6   "designation": "Developer",
7   "salary": 1900,
8   "date_of_joining": "2026-01-23",
9   "department": "",
10  "employee_photo": "https://
11  conteudo.imguol.com.br/3f/
12  2020/11/17/
13  kid-bengala-via-instagram-1605
14  617856164_v2_4x3.jpg"
15 }
```

Body Cookies Headers (8) Test Results 200 OK 93 ms 773 B Save Response

JSON Preview Visualization

```
1 {
2   "data": {
3     "getAllEmployees": [
4       {
5         "id": "69757aee88996ea6f7952a50",
6         "first_name": "Breno",
7         "last_name": "Lopes Mafra",
8         "email": "breno@lopesmafra.com",
9         "employee_photo": "https://res.cloudinary.com/dm24ql461/image/upload/v1769306862/
10        comp3133_assignment1/c8duksysibqurluyumpv.jpg"
11       },
12       {
13         "id": "6975856a88996ea6f7952a58",
14         "first_name": "Breno",
15         "last_name": "Lopes Mafra",
16         "email": "blopesmafra.com",
17       }
18     ]
19   }
20 }
```

Get employee by ID:

comp3133_101485572_assignment1 New Import

POST SignUp POST Login POST Add new GET Untitled R POST Get all er POST Get Emp + No environment

Employee / Get Employee By ID Save Share

POST http://localhost:4000/graphql Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL Auto Fetch Schema Fetched

QUERY

```
1 query GetEmp($id: ID!) {
2   getEmployeeById(id: $id) {
3     first_name
4     designation
5   }
6 }
```

GRAPHQL VARIABLES

```
1 {
2   "id":
3     "69757aee88996eaf7952a50"
}
```

Body Cookies Headers (8) Test Results 200 OK 86 ms 345 B Save Response

{ } JSON Preview Visualization

```
1 {
2   "data": {
3     "getEmployeeById": {
4       "first_name": "Breno",
5       "designation": "Developer"
6     }
7   }
8 }
```

Trying to get employee by an inexistent ID:

comp3133_101485572_assignment1 New Import

POST SignUp POST Login POST Add new GET Untitled R POST Get all er POST Get Emp + No environment

Employee / Get Employee By ID Save Share

POST http://localhost:4000/graphql Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL Auto Fetch Schema Fetched

QUERY

```
1 query GetEmp($id: ID!) {
2   getEmployeeById(id: $id) {
3     first_name
4     designation
5   }
6 }
```

GRAPHQL VARIABLES

```
1 {
2   "id":
3     "69757aee88996eaf7952a50"
}
```

Body Cookies Headers (8) Test Results 200 OK 13 ms 442 B Save Response

{ } JSON Preview Visualization

```
1 {
2   "errors": [
3     {
4       "message": "Cast to ObjectId failed for value \"69757aee88996eaf7952a50\" (type string)
5       at path \"_id\" for model \"Employee\""
6     }
7   ],
8   "data": {
9     "getEmployeeById": null
10  }
}
```

Updating employee by ID:

comp3133_101485572_assignment1 New Import

POST SignU POST Loglr POST Add r GET Unutilr POST Get al POST Get Er POST Upda + No environment

Employee / Update employee by ID Save Share

POST http://localhost:4000/graphql Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL Auto Fetch Schema Fetched

QUERY

```
1 mutation UpdateEmp($id: ID!, $salary: Float,
2   $designation: String, $department: String) {
3   updateEmployeeById(
4     id: $id,
5     salary: $salary,
6     designation: $designation,
7     department: $department
8   ) {
9     id
10    first_name
11    salary
12    designation
13    department
14  }
15 }
```

GRAPHQL VARIABLES

```
1 {
2   "id":
3     "69757aee88996ea6f7952a50",
4   "salary": 3000,
5   "designation": "Full-Stack
6     Developer",
7   "department": "Engineering"
8 }
```

Body Cookies Headers (8) Test Results 200 OK 100 ms 462 B Save Response

JSON Preview Visualization

```
1 {
2   "data": {
3     "updateEmployeeById": {
4       "id": "69757aee88996ea6f7952a50",
5       "first_name": "Breno",
6       "salary": 3000,
7       "designation": "Full-Stack Developer",
8       "department": "Engineering",
9       "updated_at": "1769309787981"
10    }
11  }
12 }
```

Search employee by designation or department:

comp3133_101485572_assignment1 New Import

POST Add GET Unutilr POST Get POST Get POST Upc DEL Delet POST Sea + No environment

Employee / Search Employee by Designation or Department Save Share

POST http://localhost:4000/graphql Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL Auto Fetch Schema Fetched

QUERY

```
1 query Search($designation: String, $department: String) {
2   searchEmployee(designation: $designation, department:
3     $department) {
4     id
5     first_name
6     last_name
7     designation
8     department
9   }
10 }
```

GRAPHQL VARIABLES

```
1 {
2   "designation": "Full-Stack
3     Developer",
4   "department": "Engineering"
5 }
```

Body Cookies Headers (8) Test Results 200 OK 79 ms 443 B Save Response

JSON Preview Visualization

```
1 {
2   "data": {
3     "searchEmployee": [
4       {
5         "id": "69757aee88996ea6f7952a50",
6         "first_name": "Breno",
7         "last_name": "Lopes Mafra",
8         "designation": "Full-Stack Developer",
9         "department": "Engineering"
10      }
11    ]
12  }
13 }
```

Trying to get an employee by an inexistent designation or department:

The screenshot shows the GraphQL Playground interface. The left sidebar contains a collection named "Employee" with several queries. The "GET Search Employee by Designation ..." query is selected. The main editor shows a POST request to `http://localhost:4000/graphql` with the following query:

```
1 query Search($designation: String, $department: String) {  
2   searchEmployee(designation: $designation, department:  
3     $department) {  
4     id  
5     first_name  
6     last_name  
7     designation  
8     department  
9   }  
}
```

The GraphQL variables are set to:

```
1 {  
2   "designation": "Marketing",  
3   "department": "Engineering"  
4 }
```

The response status is 200 OK. The response body is:

```
{  
  "data": {  
    "searchEmployee": []  
  }  
}
```

Delete employee by ID:

The screenshot shows the GraphQL Playground interface. The left sidebar contains a collection named "Employee" with several queries. The "DEL Delete Employee By ID" query is selected. The main editor shows a POST request to `http://localhost:4000/graphql` with the following query:

```
1 mutation DelEmp($id: ID!) {  
2   deleteEmployeeById(id: $id)  
3 }
```

The GraphQL variables are set to:

```
1 {  
2   "id":  
3     "69757aee88996ea6f7952a50"  
4 }
```

The response status is 200 OK. The response body is:

```
{  
  "data": {  
    "deleteEmployeeById": "Employee deleted successfully"  
  }  
}
```

Trying to delete an employee by wrong ID:

The screenshot shows a REST client interface with a collection named 'comp3133_101485572_assignment1'. The 'Employee' collection is expanded, showing a 'DELETE Employee By ID' endpoint. The request is a POST to 'http://localhost:4000/graphql' with the following GraphQL query:

```
1 mutation DelEmp($id: ID!) {  
2   deleteEmployeeById(id: $id)  
3 }
```

The GraphQL variables are set to:

```
1 {  
2   "id":  
3     "69757aee88996ea6f952a50"  
}
```

The response is a 200 OK with the following JSON body:

```
1 {  
2   "errors": [  
3     {  
4       "message": "Cast to ObjectId failed for value \"69757aee88996ea6f952a50\" (type string)  
5         at path \"_id\" for model \"Employee\""  
6     }  
7   ],  
8   "data": {  
9     "deleteEmployeeById": null  
10  }  
}
```

4. Project ZIP File:

(DONE, attached in D2L submission)

5. GitHub Project Link:

https://github.com/BrenoMafra13/Breno_COMP3133_101485572_Assignment1

6. Sample User Detail:

```
7. mutation CreateUser($username: String!, $email: String!, $password: String!) {  
8.   signup(username: $username, email: $email, password: $password) {  
9.     id  
10.    username  
11.    email  
12.  }  
13. }  
14.  
{  
  "username": "breno_mafra",  
  "email": "breno@georgebrown.ca",  
  "password": "12345678"  
}
```

15. Comments:

This time I have chosen to not host the application, but you can find everything working in this file and the D2L submission files.